

Search Algorithms in Artificial Intelligence Research and Industrial Applications

Group Name: RoboNavigators

Group Members:	MMU ID:
Ghulam Abbas Ahmed Akhtar	22506718
Muhammad Faizan Ahmed	25937922
Muhammad Saad	25948160
Aung Nyan Htun	25930702

Contents

Section 1: Group	2
Environment and algorithm design	2
Results	3
Evaluation and Comparison	4
Discussion	6
References	7
Section 2: Individual	8
Introduction	8
AI Application Areas	8
Chosen Domain: Autonomous Driving	9
A* Algorithm	10
Advantages	10
Limitations	10
Mitigation through A* variants	11
RRT Algorithm	12
Advantages	12
Limitations	12
Mitigation through RRT Variants	12
A* and RRT Comparison	13
Conclusion	13
References	14

Section 1: Group

Environment and algorithm design

We have chosen a warehouse environment for this project to model a realistic industrial navigation scenario influenced by Amazon Ltd, which uses similar robots in its fulfilment centres. We got the platform structure idea from their warehouses, where they have narrow aisles, storage racks, tables, and various other obstacles that provide a challenging yet practical testbed for evaluating the efficiency, optimality, and scalability of classical search algorithms.

The mobile robot (AGV) is modelled as a single agent operating in a two-dimensional warehouse environment. The robot occupies a 1 x 1 grid cell; it represents a compact robot used for navigation and item transportation. The robot's state is defined by its grid coordinates (x, y), which indicate its current position within the environment.

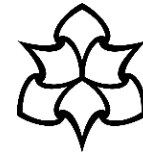
The robot follows a 4n model, which allows it to move up, down, left and right, Russell, S.J. et al. (2022). Each action moves the robot by one grid cell and incurs a uniform cost of 1; diagonal moves are not permitted. This action model ensures a fair and consistent comparison across all implemented search algorithms, including Breadth-First Search (BFS), Depth-First Search (DFS), Uniform Cost Search (UCS), and A* Search.

The environment used is a custom-designed warehouse layout created for this project to simulate a realistic industrial navigation scenario. The environment is made on a grid-based map with the following specifications:

- Grid dimensions: 16x30 grid map
- Image resolution: 800x1500 pixels
- Grid cell size: 50 x 50 pixels
- Workspace (excluding walls): 14x28 (392 grid cells)
 - o Obstacles: 86
 - o Free: 306

Each grid cell corresponds to a navigable area for the robot; the environment is represented using an occupancy grid, where:

- 0 indicates a free space
- 1 indicates obstacles



Each obstacle occupies a predefined number of grid cells, detailed in the table below:

Warehouse Objects and Obstacles	
Object/Obstacle	Size (Grid cells)
Table	4 x 2
Cone	1 x 1
Pallet	2 x 2
Rack	1 x 4
Small barrier	1 x 2
Large barrier	1 x 4
Barrel	1 x 1

The robot's initial and goal states are not fixed, allowing the user to change the default set values. The application will return an error if the coordinates entered are outside the environment boundary or are placed on a grid cell with an obstacle.

Results

Algorithms implemented based on 'tree search':

- a) T-BFS: Path found (see Fig. T-BFS). Time = 0.105317 s, Visited = 2 237, Cost = 35, Memory = 85.086 KB.
- b) T-DFS: Path found (see Fig. T-DFS). Time = 0.003603 s, Visited = 38, Cost = 37, Memory = 9.039 KB.
- c) T-UCS: No path found (terminated before solution; see discussion in Evaluation).
- d) T-A*: No path found (terminated before solution; see discussion in Evaluation).

Algorithms implemented based on 'graph search':

- a) G-BFS: Path found (see Fig. G-BFS). Time = 0.014538 s, Visited = 291, Cost = 35, Memory = 35.5 KB.
- b) G-DFS: Path found (see Fig. G-DFS). Time = 0.002319 s, Visited = 38, Cost = 37, Memory = 9.039 KB.
- c) G-UCS: Path found (see Fig. G-UCS). Time = 0.017269 s, Visited = 291, Cost = 35, Memory = 35.391 KB.
- d) G-A*: Path found (see Fig. G-A*). Time = 0.019912 s, Visited = 147, Cost = 35, Memory = 31.117 KB.

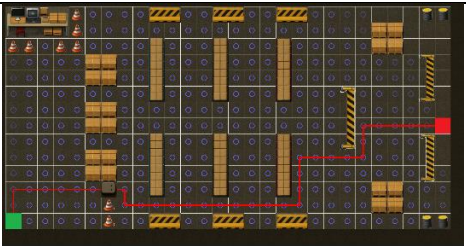
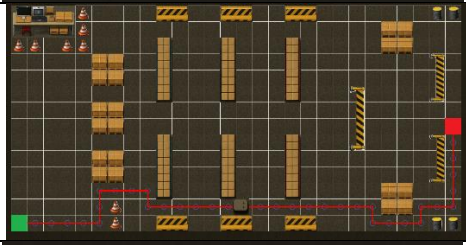
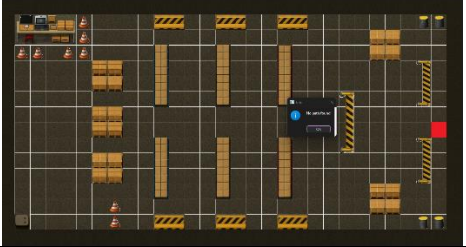
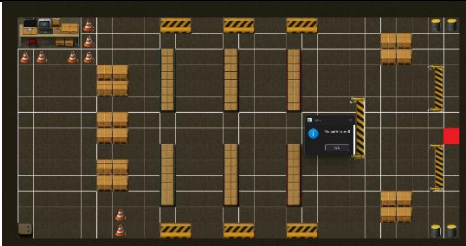
1CWK100- Assessment Report

6G7V0011 Artificial Intelligence Principles



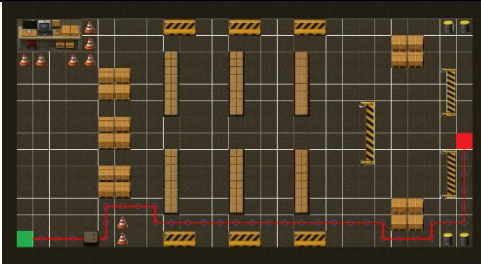
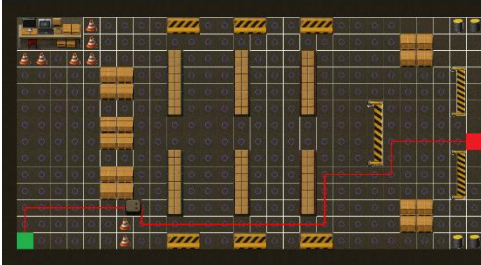
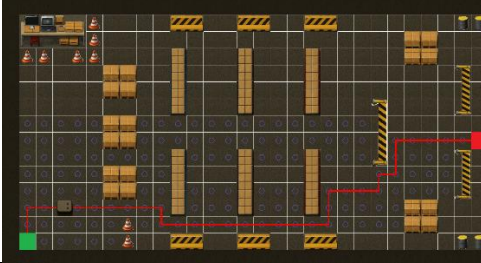
Evaluation and Comparison

For this experiment, we chose the following coordinates for the start and goal nodes: (1, 1) and (28, 7), respectively. All algorithms were compared using metrics such as processing time, solution path cost, memory storage and the number of nodes visited.

	Time (s)	Path	Cost	Storage	Nodes Visited
T-BFS	0.105317 s		35	85.086KB	2237
T-DFS	0.003603 s		37	9.039KB	38
T-UCS	N/A		N/A	N/A	N/A
T-A*	N/A		N/A	N/A	N/A
G-BFS	0.014538 s		35	35.5KB	291

1CWK100- Assessment Report

6G7V0011 Artificial Intelligence Principles

G-DFS	0.002319 s		37	9.039KB	38
G-UCS	0.017269 s		35	35.391 KB	291
G-A*	0.019912 s		35	31.117KB	147

For tree-search algorithms, we concluded that DFS was both the quickest and required the least memory for this scenario. It also expanded only 38 nodes as it explored deeply along one branch and stopped as soon as it found a goal state. BFS provided the most optimal solution with a cost of 35 (as opposed to DFS with 37), however, at the massive cost of 2237 expanded nodes and 85.086KB of memory. UCS and A* both failed to find a solution within the allotted iteration limit (10000).

Among graph-search algorithms, A* achieved the best overall performance by expanding the most promising nodes first. Of all the optimal algorithms at a cost of 35, it expanded the fewest nodes (147) and required the least memory (31.117KB). This efficiency is due to the Manhattan-distance heuristic, which effectively guides the search toward the goal while avoiding unnecessary exploration (Ai, 2024b). BFS and UCS also return optimal solutions, but their numbers of expanded nodes (291) and memory consumed (around 35.5KB) are both larger than those of A*. The graph-search version of DFS was also very fast and low on memory, but like its tree-search version, it found a suboptimal solution with a cost of 37.

For the same algorithm, the graph-search version is far more efficient and more robust than the tree-search version. Most notably, in BFS, the number of nodes visited drops from 2237 in tree search to 291 in graph search, as it prevents revisiting the nodes. The

difference is more striking for UCS and A*, where graph and tree search differ in their algorithmic behaviour; they were able to find optimal solutions in graph search mode but failed in tree search due to excessive node revisiting, exceeding the predefined iteration limit.

Overall, after running different scenarios, we concluded that graph-search A* is the best choice, as it kept the number of nodes expanded to a minimum and required a reasonable amount of memory while usually finding an optimal solution. A* graph search offers the best balance between performance and solution quality for this warehouse navigation task.

Discussion

The choice of a search algorithm for a real-world problem depends on many factors, including the level of optimality required, the computational power available, and the size of the environment, Russell, S.J. et al. (2022). Both UCS and A* are most used for problems where the environment is static, well-defined, and we need optimal paths. Their effectiveness also depends on proper duplicate-state handling and sufficient memory availability. DFS and BFS are also good choices, but only for a small-scale environment and when optimality is not essential.

In industrial robotics applications, like warehouse automation, the workspace may be large-scale and contain numerous obstacles. The results showed that tree-search methods are inefficient under such conditions, while graph-search methods are far more appropriate for real-world navigation tasks.

However, if obstacles in the environment were to be dynamic, the static search algorithms implemented in this project would not be sufficient. Based on our understanding, one way to improve this is to incorporate replanning techniques, such as repeatedly invoking A* when changes are detected. However, these improvements come with limitations, such as an increased need for computational resources, which makes it much more expensive to run. Alternatively, incremental search algorithms like D* or D* Lite update previous search efforts to efficiently replan existing paths to the goal after each environmental change, making them a viable alternative (Tang and Ma, 2021). These approaches not only allow the robot to adjust its path in real time but are also computationally efficient for use in real-world environments, where conditions may be dynamic and unpredictable.

1CWK100- Assessment Report

6G7V0011 Artificial Intelligence Principles



References

Ai, T.A. (2024b) A* search algorithm (A* algorithm in AI). Available at: <https://www.appliedaicourse.com/blog/a-star-algorithm-in-ai/>.

Tang, Z. and Ma, H. (2021) 'An overview of path planning algorithms,' IOP Conference Series Earth and Environmental Science, 804(2), p. 022024. Available at: <https://doi.org/10.1088/1755-1315/804/2/022024>.

Russell, S.J. et al. (2022) *Artificial intelligence: a modern approach. Fourth edition*. Global edition. Harlow, United Kingdom: Pearson.

Section 2: Individual

Introduction

In this rapidly changing world, AI is significantly contributing to the advancement of the technological landscape. In general, AI refers to the study of intelligent agents that perceive their environment and perform actions accordingly (Russell & Norvig, 2021). It is categorised into three types: Narrow AI, designed to perform specific tasks; General AI, hypothetical AI that matches human-level intelligence; and Superintelligent AI, hypothetical AI that is superior to human intelligence. Current AI systems fall under the umbrella of Narrow AI.

AI is further divided into subfields that contribute to different domains. Machine learning (ML) focuses on learning from data, getting insights, and making informed decisions. Computer vision (CV) enables machines to perceive the environment as humans do. Natural language processing (NLP) enables machines to understand, interpret, and generate human language in a meaningful, context-aware manner. Deep learning, a subfield of machine learning, uses neural networks to extract insights from large-scale data.

AI Application Areas

From education to healthcare, agriculture to finance, manufacturing to transportation, AI has transformed almost every sector in no time. AI supports the education sector by creating personalised lessons, generating questionnaires, and grading assessments fairly. In healthcare, AI-enabled devices are used for early disease detection, reading scans, and suggesting treatments. In the agricultural industry, it helps detect plant diseases, monitor weather conditions, save water through smart irrigation, and support farmers in making better decisions. AI benefits the finance sector by helping with budgeting and planning, predicting market trends, detecting fraud, and supporting better financial decisions. In the manufacturing domain, AI predicts machine failures before breakdowns, checks product quality, and detects anomalies, thereby reducing manufacturing costs and material waste and promoting sustainable production.

Apart from generic AI applications, as mentioned earlier, certain specific domains require continuous autonomous decision-making while operating in a physical environment. In autonomous driving, the vehicle must perceive the environment, plan obstacle-free trajectories, and deal with dynamic objects in real time, which requires effective path planning and better decision-making. AI plays a key role in enabling vehicles to perceive their environment, plan safe trajectories, and navigate through the real-world environment (Zade, N. *et al.*, 2024, *p.1*). AI has also transformed the human-robot interaction (HRI) domain by enabling robots to perform context-aware decision-making and adaptive movement when working with humans (Lin and Qi, 2024). AI supports digital twin solutions to simulate system

behaviour and test alternative operational scenarios before actual implementation. This highlights how AI is advancing these domains by introducing robust planning and decision-making.

Chosen Domain: Autonomous Driving

Among the previously mentioned domains, autonomous driving requires sophisticated use of AI for planning safer routes to the destination. Autonomous vehicles operate in a dynamic environment where surrounding conditions are changing continuously, and decisions must be made in real time. In order to navigate safely and effectively in a constrained environment, continuous path and trajectory planning is required. The following section discusses various path-planning algorithms widely used to address these challenges in autonomous driving.

In autonomous driving, the path-planning task is divided into local and global planning. Global planning focuses on generating the road-level path, while local planning generates a short-term trajectory which considers vehicle dynamics (Teng *et al.*, 2023, p. 2). For global path planning, search-based algorithms such as A* or Dijkstra are widely used to compute the shortest path between locations. For local planning tasks where the environment is complex and optimal control is required, sampling-based algorithms such as Rapidly-exploring Random Trees (RRT) and its variants are preferred over search-based algorithms because they can efficiently explore high-dimensional state spaces (Yurtsever *et al.*, 2020, p. 18). Recent research also highlights growing interest in learning-based planning methods, such as imitation learning and reinforcement learning, which directly learn driving behaviours from data. However, classical search-based algorithms remain central in most practical autonomous driving systems due to their reliability, interpretability, and suitability for safety-critical applications (Teng *et al.*, 2023b, pp. 1–2).

Search-based algorithms follow a systematic approach to path planning and decision-making by exploring various states and actions to find the optimal solution. Due to the complex nature of physical driving scenarios, a single algorithm is not enough to address these challenges. Therefore, multiple search algorithms are used in conjunction to deal with these problems and enable autonomous vehicles to navigate safely.

In the following section, A* and RRT search algorithms for planning and navigation in autonomous driving will be discussed. The selected algorithms use either heuristic-based or sampling-based search techniques to find the end path. Recent research in autonomous path and trajectory planning focuses on using optimised variants of these algorithms. Each algorithm will be evaluated based on its strengths, limitations, and suitability for different planning scenarios.

A* Algorithm

A* is a heuristic-based search algorithm used for planning paths for autonomous vehicles, developed by researchers of Stanford Research Institute (SRI) in 1968. In an A* search algorithm, the cost $f(n)$ is computed for all the adjacent nodes from the current node, which is as follows:

$$f(n) = g(n) + h(n)$$

where,

$g(n)$: cost from the start node to the current node

$h(n)$: estimated cost from the current node to the goal node

The A* algorithm selects the next node from the adjacent nodes with the least cost and continues searching until the goal node is found. Heuristic function $h(n)$ is calculated using either the Manhattan distance (suitable for grid-based environments) or the Euclidean distance (suitable for continuous spaces).

Advantages

A* algorithm performs well in static and well-known environments. It provides goal-directed search by combining actual cost and the estimated cost to the goal. Solution optimality is guaranteed if the heuristic is admissible and consistent. It reduces unnecessary node expansion, which makes it computationally efficient. Its algorithm structure is simple and easy to implement (Paliwal, 2023; Luo *et al.*, 2025, p. 2; Sharan *et al.*, 2025, p. 4).

Limitations

The traditional A* algorithm has various limitations. It works well when the environment is static, which is certainly not the case in physical driving scenarios, where the surrounding environment is constantly changing. The computed route is not smooth and contains sharp turns, which do not replicate the actual vehicle movement in the physical environment. It heavily relies on heuristic quality to find the optimal route and is also computationally expensive (Paliwal, 2023; Zheng *et al.*, 2023; Luo *et al.*, 2025). The following section describes how these limitations were addressed by improving the traditional A* algorithm.

Mitigation through A* variants

Over time, a variety of improvements to the classical A* algorithm have been proposed to mitigate its challenges. Some widely used variants are defined in detail, including the challenges they address and their own limitations.

Hybrid A* is one of the improved variants of the A* algorithm, which operates in a continuous state space. It considers vehicle kinematic constraints and generates smoother trajectories for autonomous driving. Locally optimal solution and higher computational cost are the limitations of this optimised algorithm (Shang *et al.*, 2024, p. 1).

Improved A* is another variant of the classical A* algorithm, which provides a shorter optimal path and guided search capability when navigating around obstacles. It works by initially making a straight-line connection between the start and goal nodes and then replanning the path around the obstacles along that straight line. Increased computational resources and a slower response rate limit the capability of this optimised approach (Paliwal, 2023, p. 3).

Anytime Repairing A* (ARA*) is used in environments where the optimal solution cannot be computed due to time constraints. It provides real-time response and fast initial solutions. First, it computes a highly sub-optimal solution quickly and then keeps improving it until the time reaches zero. It's efficient because it uses previous search results to avoid repeated computation. The limitation of this updated approach is the sub-optimal solution and the quality, which improves over time (Paliwal, 2023, p. 3).

Dynamic A* Lite (D* Lite) is a replanning-based A* algorithm used in situations where changes are occurring in the graph. It handles newly detected obstacles and constantly replans the path while navigating in the search space. The main hindrance to this approach is the algorithm's complexity (Paliwal, 2023, p. 4).

Anytime Dynamic A* (AD*) makes use of the capabilities of both ARA* and D* Lite algorithms. It is useful in dynamic environments, providing fast replanning and adaptive decision-making. This algorithm utilises higher computational resources due to its complex nature (Paliwal, 2023, pp. 4–5).

AD* and Hybrid A* are the two most efficient variants of the A* algorithm; the former can work in dynamic environments and make instant decisions when new obstacles appear, while the latter keeps an eye on the physical vehicle dynamics constraints and generates uninterrupted trajectories. These variants addressed the limitations of the classical A* algorithm, particularly in terms of search efficiency and practical applicability.

RRT Algorithm

RRT is one of the widely used algorithms for autonomous path planning. It is a sampling-based path-planning algorithm that gradually constructs a tree through random sampling in the search space and iteratively expands it until the goal is reached. It is suitable for high-dimensional and obstacle-dense search environments (Wu *et al.*, 2025, p. 2; Ying, 2025, p. 1).

Advantages

RRT is suitable for an unknown or partially known environment. It efficiently explores high-dimensional search spaces via random sampling, enabling fast pathfinding in such scenarios. It is easily scalable to complex, constrained environments due to its easier implementation and simple algorithmic structure (Feraco *et al.*, 2020c, p. 1; Ying, 2025, p. 1).

Limitations

The traditional RRT algorithm has several limitations. The optimal solution is not guaranteed due to the random sampling of the generated tree. It also does not account for vehicle kinematic constraints, and the computed route is not smooth with sharp turns, which do not correspond to the actual vehicle movement in the physical autonomous driving scenarios (Ren, Chen and Fu, 2021, p. 1; Zhang *et al.*, 2021, p. 1; Ying, 2025, p. 2).

Mitigation through RRT Variants

Expanding on the traditional RRT algorithm, various improvements have been made to enhance planning efficiency, path quality, and practical applicability in complex environments. Some widely used variants are defined in detail, including the challenges they address and their own limitations.

Bidirectional RRT is one of the earliest variants of the classical RRT algorithm, used in environments where the start and goal nodes are located in narrower spaces. It starts growing two trees, one from the start node and the other from the goal node, simultaneously, until they meet at a common point to form a solution path, thereby improving path planning efficiency. The solution remains suboptimal, limiting the algorithm's capabilities (Zhang *et al.*, 2021, p. 1).

Improved RRT is another variant of the RRT algorithm where target-based sampling is used to expand the random tree to the target point quickly. The pruning method and the B-spline curve are used to improve the overall path quality. Algorithm complexity is the main limitation of this approach (Zhu, Lian and Gui, 2023, p. 1).

Heuristic-guided RRT uses a heuristic alongside the existing RRT tree expansion capability. The heuristic function helps the tree guide itself towards the goal, reducing randomness. The guidance towards the target point depends on the heuristic's quality, which can divert the overall search if it is not optimal (Ying, 2025, p. 3).

RRT* is the updated version of the traditional RRT algorithm used for complex environments where path optimality is required. It uses a rewiring mechanism to gradually optimise the path and guarantees asymptotic optimality, meaning the solution will converge to the target point as the number of samples approaches infinity. Slow convergence rate and higher computational costs are the major drawbacks of this approach (Wu *et al.*, 2025, p. 2).

Neural RRT* is the neural network-based variant of the RRT algorithm that uses CNN-guided sampling to guide the tree to the target point. It provides a higher convergence rate and improves the overall optimality of the path. The drawback of this technique is that it requires extensive training data (Wang *et al.*, 2020b, pp. 5–6).

These RRT variants aim to mitigate the limitations of the traditional RRT algorithm by improving solution optimality, path quality, and planning efficiency.

A* and RRT Comparison

A* is a graph-based search algorithm, providing heuristic-guided exploration, whereas RRT is a sampling-based planning algorithm focusing on random exploration of the search space. A* guarantees an optimal solution subject to heuristic quality, but RRT is not guaranteed to find an optimal path due to its random exploration strategy. A* produces a polyline path with sharp turns, while RRT often results in irregular or discontinuous paths, requiring the need for a smoothing method after both algorithms. The traditional variant of both A* and RRT algorithms does not consider vehicle kinematics, while the improved variants take note of it while navigating through the complex, obstacle-dense environments. A* is suitable for small-scale and less-dense environments, while RRT is well-suited for high-dimensional and complex environments.

Conclusion

To conclude, this section examined the role of AI in autonomous driving, particularly focusing on search-based planning algorithms. After analysing A* and RRT algorithms and their variants, it was shown that different algorithms address different planning challenges. A* provides structured, reliable solutions for path planning, while RRT offers greater flexibility in complex, high-dimensional environments. The development of algorithm variants highlights ongoing efforts to improve efficiency, feasibility, and practical applicability. Overall, effective autonomous navigation depends on selecting appropriate planning algorithms for the given search space.

References

Feraco, S. *et al.* (2020c) 'A local trajectory planning and control method for autonomous vehicles based on the RRT algorithm', *2020 AEIT International Conference of Electrical and Electronic Technologies for Automotive (AEIT AUTOMOTIVE)* [Preprint]. Available at: <https://doi.org/10.23919/aeitautomotive50086.2020.9307439>.

Luo, S. *et al.* (2025) 'Autonomous Navigation Path Planning Based on Improved A* Algorithm', *2025 44th Chinese Control Conference (CCC)*, pp. 4758–4763. Available at: <https://doi.org/10.23919/ccc64809.2025.11179707>.

Paliwal, P. (2023) 'A Survey of A-Star Algorithm Family for Motion Planning of Autonomous Vehicles', *2023 IEEE International Students' Conference on Electrical, Electronics and Computer Science (SCEECS)*, pp. 1–6. Available at: <https://doi.org/10.1109/sceecs57921.2023.10063063>.

Ren, P., Chen, S. and Fu, H. (2021) 'Intelligent Path Planning and Obstacle Avoidance Algorithms for Autonomous Vehicles Based on Enhanced RRT Algorithm', *2021 6th International Conference on Communication and Electronics Systems (ICCES)*, pp. 1868–1871. Available at: <https://doi.org/10.1109/icces51350.2021.9489113>.

Russell, S. and Norvig, P. (2021) *Artificial Intelligence: A Modern Approach, Global Edition*. Pearson Higher Ed.

Shang, G. *et al.* (2024) 'Improved Hybrid A* algorithm for Autonomous Driving Path Planning', *2024 7th International Conference on Computer Information Science and Application Technology (CISAT)*, pp. 232–237. Available at: <https://doi.org/10.1109/cisat62382.2024.10695194>.

Sharan, S. *et al.* (2025) 'Comparison of Standard Ant Colony Optimization, Dijkstra and A* Algorithms for Path Planning of Robot in an Indoor Environment', *2025 9th International Conference on Mechanical Engineering and Robotics Research (ICMERR)*, pp. 10–16. Available at: <https://doi.org/10.1109/icmerr64601.2025.10949924>.

Teng, S. *et al.* (2023) 'Motion Planning for Autonomous Driving: the state of the art and future Perspectives', *IEEE Transactions on Intelligent Vehicles*, 8(6), pp. 3692–3711. Available at: <https://doi.org/10.1109/tiv.2023.3274536>.

Wang, J. *et al.* (2020b) 'Neural RRT*: Learning-Based Optimal Path Planning', *IEEE Transactions on Automation Science and Engineering*, 17(4), pp. 1748–1758. Available at: <https://doi.org/10.1109/tase.2020.2976560>.

Wu, J. *et al.* (2025) 'Floyd-Assisted RRT* for Path Planning in Static Environment', *2025 5th International Conference on Artificial Intelligence, Big Data and Algorithms (CAIBDA)*, pp. 955–959. Available at: <https://doi.org/10.1109/caibda65784.2025.11183209>.

Ying, Y. (2025) 'An Improved RRT* Algorithm for Efficient Path Planning Integrating Heuristic Functions and Attractive Potential Fields', *2025 International Conference on Digital Analysis and Processing, Intelligent Computation (DAPIC)*, pp. 807–812. Available at: <https://doi.org/10.1109/dapic66097.2025.00153>.

Yurtsever, E. *et al.* (2020) 'A survey of Autonomous Driving: Common practices and Emerging technologies', *IEEE Access*, 8, pp. 58443–58469. Available at: <https://doi.org/10.1109/access.2020.2983149>.

Zade, N. *et al.* (2024) 'The Future of Driving: A Review on the Combination of Artificial Intelligence and Autonomous Vehicles', *2024 2nd DMIHER International Conference on Artificial Intelligence in Healthcare, Education and Industry (IDICAIEI)*, pp. 1–6. Available at: <https://doi.org/10.1109/idicaiei61867.2024.10842905>.

Zhang, Y. *et al.* (2021) 'An Improved Dynamic Step Size RRT Algorithm in Complex Environments', *2021 33rd Chinese Control and Decision Conference (CCDC)*, pp. 3835–3840. Available at: <https://doi.org/10.1109/ccdc52312.2021.9602069>.

Zheng, H. *et al.* (2023) 'The Navigation Based on Hybrid A Star and TEB Algorithm Implemented in Obstacles Avoidance', *2023 29th International Conference on Mechatronics and Machine Vision in Practice (M2VIP)*, pp. 1–6. Available at: <https://doi.org/10.1109/m2vip58386.2023.10413435>.

Zhu, J., Lian, X. and Gui, Z. (2023) 'Research on Global Path Planning System of Driverless Car Based on Improved RRT Algorithm', *2023 International Conference on Data Science & Informatics (ICDSI)*, pp. 260–263. Available at: <https://doi.org/10.1109/icdsi60108.2023.00056>.